

Lecture 5

간단한 아케이드 게임을 위한
플레이어-적 비행체 제어
Unity를 이용한 2D 게임프로그래밍

강영민

동명대학교 게임공학과

목차

- 1 학습목표
- 2 프로젝트 설정
- 3 Prefab 제작
- 4 New Input System
- 5 플레이어 이동
- 6 경계 설정
- 7 적의 경로 이동 – Waypoint
- 8 ScriptableObject – WaveConfigSO
- 9 적의 움직임 – Pathfinder
- 10 적 인스턴스 생성 – EnemySpawner
- 11 Coroutine을 이용한 시차 생성
- 12 여러 Wave와 무한 반복
- 13 정리

◎ 이번 강의에서 배울 내용

- 전통적인 아케이드 슈팅 게임 프로젝트를 만들어 본다
- 사용자 입력을 처리하기 위한 **New Input System**을 사용하자
- **Prefab**에 대해 이해하자
- Prefab 객체의 **생성과 소멸**
- 루프를 이용하여 적의 움직임 제어
 - Waypoint를 따라 움직이기
 - 여러 객체가 해당 움직임을 따르게 하기
 - 다수의 경로(Path)를 준비하여 동작시키기
 - 적기 생성이 끝나버리지 않게 무한히 반복시키기

프로젝트 생성



Space Shooter 프로젝트

- Unity Hub에서 **Universal 2D** 템플릿 선택
- 프로젝트 이름: SpaceShooter
- Unity 버전: 6000.0.41f1 LTS



해상도 설정

- Game 뷰에서 해상도를 **768×1366**으로 변경
- 세로형 모바일 레이아웃
- Custom 해상도 추가 가능



왜 세로 해상도?

전통적인 아케이드 슈팅 게임은 세로 화면을 사용합니다. 적이 위에서 아래로 내려오고, 플레이어가 아래에서 조작하는 구조입니다.

게임 리소스 준비

Kenney.nl 에셋

- **kenney.nl**에서 “Space Shooter Redux” 검색
- 무료 게임 에셋 제공 사이트
- 다운로드 후 AssetPack 폴더를 Unity 프로젝트에 드래그 & 드롭

Texture Type 확인

Import한 PNG 파일들의 Texture Type이 **Sprite (2D and UI)**로 설정되어 있는지 확인하세요. Inspector에서 여러 파일을 선택하여 한꺼번에 확인할 수 있습니다.

에셋 폴더 구조

```
Assets/AssetPack/  
  Backgrounds/  
  Bonus/  
  PNG/  
  Damage/  
  Effects/  
  Enemies/  
  Lasers/  
  Meteors/  
  Parts/  
  Power-ups/  
  UI/  
  playerShip*.png  
  ufo*.png  
  Spritesheet/  
  Vector/
```

Prefab이란?

Prefab 개념

Prefab(프리팹)은 게임 오브젝트의 **템플릿(틀)**입니다.

- 한 번 만들어 두면 여러 번 **재사용** 가능
- 원본(Prefab)을 수정하면 모든 인스턴스에 반영됨
- 코드에서 `Instantiate()`로 런타임에 생성 가능
- Hierarchy의 오브젝트를 Project 창의 폴더로 드래그하면 Prefab이 됨

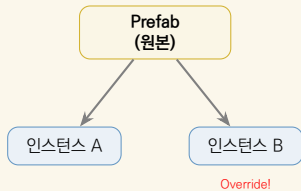
Prefabs 폴더

Assets 아래에 **Prefabs** 폴더를 만들어 Prefab들을 정리합니다.

주목: Prefab과 인스턴스의 관계

★ 주목 - Prefab, 인스턴스, 그리고 Override

- **Prefab** = 설계도, **인스턴스** = 찍어낸 실제 오브젝트
- Prefab 수정 → 모든 인스턴스에 **자동 반영**
- 인스턴스에서 값을 바꾸면 **Override**(재정의)됨
- 우클릭 → Revert로 원본 복원 가능



★ 주목 - 왜 빈 GameObject에 스프라이트를 자식으로?

- **스크립트, Collider, Rigidbody** 등 컴포넌트를 부모에 추가하기 위해
- 스프라이트 교체 시 부모 설정 유지 / 여러 스프라이트를 자식으로 묶기 가능

Prefab 제작 1 – Player

플레이어 Prefab 만들기

- 1 Hierarchy에서 빈 **GameObject** 생성 → 이름을 “Player”로 변경
- 2 PNG 폴더의 플레이어 스프라이트를 Player의 **자식(Child)**으로 드래그
- 3 Assets에 **Prefabs** 폴더 생성
- 4 Hierarchy의 “Player”를 Prefabs 폴더로 드래그 → **Prefab 생성 완료**

Prefab 확인

Hierarchy에서 Prefab으로 등록된 오브젝트는 **파란색**으로 표시됩니다.

Prefab 제작 2 – Enemy

적 Prefab 만들기

Player와 동일한 방식으로 “Enemy 0” Prefab을 제작합니다.

- 1 빈 GameObject 생성 → “Enemy 0”으로 이름 변경
- 2 적 스프라이트(예: `enemyRed1`)를 자식으로 드래그
- 3 Prefabs 폴더로 드래그하여 Prefab 저장

문제점과 해결

- 캐릭터가 게임 공간에 비해 너무 크다 → 카메라의 **Orthographic Size** 값을 조정
- 두 캐릭터가 같은 위치에 겹침 → Prefab의 **Position**을 변경하여 분리
- 배경 이미지가 캐릭터를 가림 → 배경의 **Sorting Order**를 -1로 설정

주목: Sorting Order와 Sorting Layer

★ 주목 - 2D에서 “앞”과 “뒤”는 어떻게 결정되는가?

2D에서는 **Sprite Renderer**의 렌더링 순서로 앞뒤가 결정됩니다.

- **Sorting Layer**: 렌더링 그룹 (예: Background, Default, Foreground)
- **Order in Layer**: 같은 Layer 안에서의 순서 (숫자가 **클수록 앞**)

★ 주목 - 렌더링 우선순위 규칙



규칙 1: Sorting Layer가 아래일수록 뒤

규칙 2: 같은 Layer면 Order가 작을수록 뒤

규칙 3: 모두 같으면 Z축 위치로 판단

배경을 **Order = -1**로 설정하면 캐릭터(**Order: 0**)보다 뒤에 그려집니다.

New Input System 설정

+ 패키지 설치

- 1 Window → Package Manager
- 2 “Input System” 검색 후 설치
- 3 설치 후 Unity 재시작이 필요할 수 있음

⚙ Player Input 컴포넌트

- 1 Prefabs 폴더에서 Player Prefab 선택
- 2 Add Component → Player Input
- 3 Create Actions... 버튼 클릭
- 4 파일명: InputActions
- 5 Prefabs 폴더에 저장

☰ Input Actions 구조

Action Maps:

- Player – 게임 플레이 입력
- UI – UI 네비게이션 입력

Player Map의 Actions:

- Move – Value 타입, **Vector2**
- Look – Value 타입, Vector2
- Fire – Button 타입

Key Bindings 설정

Move 액션 바인딩

Input Actions 에디터에서 **Move** 액션의 바인딩:

- **Left Stick [Gamepad]** – 게임패드
- **WASD [Keyboard]** – 2D Vector Composite
 - Up: W / Up Arrow
 - Down: S / Down Arrow
 - Left: A / Left Arrow
 - Right: D / Right Arrow
- **Stick [Joystick]** – 조이스틱

Player Input 동작시키기

Player Input이 동작하려면:

- 1 Player Input 컴포넌트의 **Open Input Settings** 클릭
- 2 Project Settings에서 **Create settings asset** 클릭
- 3 **InputSystemSettings** 에셋이 생성됨

Behavior 설정

Player Input의 Behavior를 **Send Messages**로 설정하면, 액션 이름 앞에 “On”을 붙인 메서드가 자동 호출됩니다.

예: Move → **OnMove()**

주목: Input System의 Behavior 방식들

★ 주목 – Player Input의 Behavior 옵션 비교

방식	설명
Send Messages	OnMove(), OnFire() 등 자동 호출. 이 강의에서 사용
Broadcast Messages	Send Messages + 자식 오브젝트 까지 호출
Invoke Unity Events	Inspector에서 이벤트를 시각적으로 연결
Invoke C# Events	코드에서 이벤트에 직접 구독 . 가장 유연하지만 복잡

★ 주목 – Send Messages 메서드 이름 규칙

액션 이름에 “On”을 접두어로 붙여야 합니다. Move → OnMove(InputValue value), Fire → OnFire(InputValue value)

이름이 틀리면 호출되지 않습니다! (오류 메시지도 없음에 주의)

Player.cs – 입력 받기

</> 입력 메시지를 가로채는 코드

```
1 using UnityEngine;
2 using UnityEngine.InputSystem; // Required for Input System
3
4 public class Player : MonoBehaviour
5 {
6     private Vector2 rawInput;
7
8     void Update() { }
9
10    void OnMove(InputValue value)
11    {
12        rawInput = value.Get<Vector2>();
13        Debug.Log("Raw Input: " + rawInput);
14    }
15 }
16
```

i 핵심 포인트

- using UnityEngine.InputSystem – Input System 네임스페이스 필수
- OnMove 메서드명은 Input Actions의 “Move” 액션에 대응
- Player.cs가 Player Prefab에 연결되어 있는지 **반드시 체크**

Player.cs – 이동 구현

</> 속도를 제어할 수 있게 해 보자

```
1 public class Player : MonoBehaviour
2 {
3     private Vector2 rawInput;
4     private float speed = 5.0f;
5
6     void Update()
7     {
8         transform.Translate(rawInput * speed * Time.deltaTime);
9     }
10
11     void OnMove(InputValue value)
12     {
13         rawInput = value.Get<Vector2>();
14     }
15 }
16
```

💡 transform.Translate

transform.Translate(방향 * 속도 * Time.deltaTime)

Time.deltaTime을 곱하면 프레임 속도에 관계없이 일정한 속도로 이동합니다.

주목: Time.deltaTime의 역할

★ 주목 - 왜 Time.deltaTime을 곱하는가?

Update()는 매 **프레임**마다 호출됩니다. 프레임 속도(FPS)는 컴퓨터 성능에 따라 다릅니다.

★ 주목 - deltaTime 없이 vs 있을 때

deltaTime 없이: pos += 5

- 60 FPS → 초당 300 이동
- 30 FPS → 초당 150 이동
- 빠른 컴퓨터에서 더 빨리 움직임!

Time.deltaTime은 직전 프레임에서 현재 프레임까지 경과한 시간(초)입니다.

이를 곱하면 “프레임당 이동”이 아닌 “초당 이동”으로 단위가 바뀝니다.

deltaTime 곱하면: pos += 5 * dt

- 60 FPS → $5 \times \frac{1}{60} \times 60 = 5/\text{초}$
- 30 FPS → $5 \times \frac{1}{30} \times 30 = 5/\text{초}$
- 어떤 컴퓨터에서든 **동일한 속도!**

Viewport와 World 좌표

田 Viewport 좌표계

- 화면을 **정규화**한 좌표
- 좌하단 = $(0, 0)$
- 우상단 = $(1, 1)$
- 해상도에 관계없이 동일한 범위

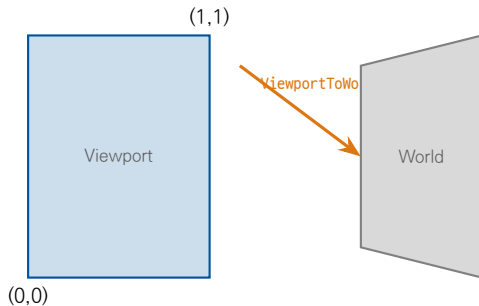
World 좌표계

게임 콘텐츠를 담은 **3차원 공간** 내의 좌표. 실제 게임 오브젝트의 `Transform.position`이 사용하는 좌표입니다.

좌표 변환

```
Camera.main.ViewportToWorldPoint(pos)
```

Viewport 좌표를 World 좌표로 변환합니다.



Player.cs – 경계 안에 움직임 제한하기

</> 화면 밖으로 나가지 않도록

```
1 public class Player : MonoBehaviour
2 {
3     private Vector2 rawInput;
4     private float speed = 5.0f;
5     Vector2 minBounds, maxBounds;
6
7     void Start()
8     {
9         Camera cam = Camera.main;
10        minBounds = cam.ViewportToWorldPoint(new Vector2(0, 0));
11        maxBounds = cam.ViewportToWorldPoint(new Vector2(1, 1));
12    }
13    void Update()
14    {
15        Vector2 delta = rawInput * speed * Time.deltaTime;
16        Vector2 newPos = (Vector2)transform.position + delta;
17        newPos.x = Mathf.Clamp(newPos.x, minBounds.x, maxBounds.x);
18        newPos.y = Mathf.Clamp(newPos.y, minBounds.y, maxBounds.y);
19        transform.position = newPos;
20    }
21    void OnMove(InputValue value)
22    {
23        rawInput = value.Get<Vector2>();
24    }
25 }
26
```

경계 설정 코드 설명

🔍 코드 분석

- ❶ **Start()**: 화면의 좌하단(0,0)과 우상단(1,1)을 World 좌표로 변환하여 경계값 저장
- ❷ **Update()**: 이동 후 위치를 `Mathf.Clamp()`로 경계 안에 제한

📖 Mathf.Clamp

`Mathf.Clamp(value, min, max)`

`value`가 `min`보다 작으면 `min`을, `max`보다 크면 `max`를 반환합니다.

이렇게 하면 플레이어가 화면 밖으로 나갈 수 없습니다.

Waypoint 시스템

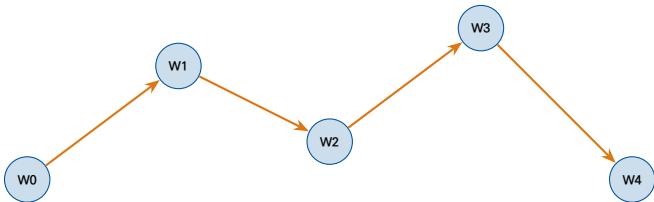
Waypoint란?

적이 이동할 **경유점(지점)**들의 모음입니다.

- 적은 Waypoint 0 → 1 → 2 → ...순서로 이동
- 마지막 Waypoint에 도달하면 소멸
- 다양한 Path를 만들어 다른 패턴의 움직임 구현

Waypoint 만들기

- 1 Empty Object로 "Path 0" 생성
- 2 Path 0의 자식으로 Waypoint (0)~(4) 생성
- 3 각 Waypoint에 아이콘 설정 (보이게)
- 4 Scene 뷰에서 원하는 위치에 배치
- 5 Waves and Paths 폴더 만들고 Path 0을 Prefab으로 저장



Destroy!

ScriptableObject란?

ScriptableObject 개념

- 데이터 컨테이너 – 게임 데이터를 코드와 분리하여 관리
- MonoBehaviour와 달리 **씬에 붙이지 않아도** 사용 가능
- [CreateAssetMenu]로 Unity 메뉴에서 에셋 생성 가능

WaveConfigSO.cs – 기본 구조

```
1 [CreateAssetMenu(fileName = "WaveConf_SO",  
2     menuName = "Scriptable Objects/WaveConf_SO")]  
3 public class WaveConfigSO : ScriptableObject  
4 {  
5     [SerializeField] Transform pathPrefab;  
6     [SerializeField] float moveSpeed = 5f;  
7 }  
8
```

생성 방법

Assets에서 우클릭 → Create → Scriptable Objects → WaveConf_SO

주목: MonoBehaviour vs ScriptableObject

★ 주목 - 두 기반 클래스의 차이

	MonoBehaviour	ScriptableObject
부착 대상	씬의 GameObject에 부착	에셋 파일로 존재
생명주기	씬과 함께 생성/파괴	프로젝트에 영구 존재
콜백 메서드	Start, Update 등 다수	Awake, OnEnable 정도
데이터 공유	인스턴스마다 독립	여러 객체가 같은 데이터 참조
용도	게임 로직, 동작	설정 데이터, 구성 정보

★ 주목 - [SerializeField]란?

private 변수를 Inspector에서 **편집 가능**하게 만드는 속성. public은 캡슐화가 깨지므로 [SerializeField] private이 Unity 권장 패턴.

WaveConfigSO – Getter 메서드

</> 데이터 접근 메서드

```
1 public Transform GetStartingWaypoint()
2 {
3     return pathPrefab.GetChild(0);
4 }
5
6 public float GetMoveSpeed()
7 {
8     return moveSpeed;
9 }
10
11 public List<Transform> GetWaypoints()
12 {
13     List<Transform> waypoints = new List<Transform>();
14     foreach (Transform child in pathPrefab)
15     {
16         waypoints.Add(child);
17     }
18     return waypoints;
19 }
20
```



주의

List<Transform>을 사용하려면 using System.Collections.Generic; 필요!

PathFinder.cs – 적이 경로를 따라 이동

</> Pathfinder 스크립트

```
1 using UnityEngine;
2 using System.Collections.Generic;
3
4 public class Pathfinder : MonoBehaviour
5 {
6     [SerializeField] WaveConfigSO waveConfig;
7     List<Transform> waypoints;
8     int waypointIndex = 0;
9
10    void Start()
11    {
12        waypoints = waveConfig.GetWaypoints();
13        transform.position = waypoints[waypointIndex].position;
14    }
15
16    void Update()
17    {
18        FollowPath();
19    }
20 }
21
```

i Pathfinder의 역할

Enemy Prefab에 이 스크립트를 연결하여 적이 경로를 따라 이동하게 합니다.

FollowPath – 경로 따라가기

</> FollowPath 메서드

```
1 void FollowPath()
2 {
3     if (waypointIndex < waypoints.Count)
4     {
5         Vector3 targetPosition = waypoints[waypointIndex].position;
6         float delta = waveConfig.GetMoveSpeed() * Time.deltaTime;
7         transform.position = Vector2.MoveTowards(
8             transform.position, targetPosition, delta);
9
10        if (transform.position == targetPosition)
11        {
12            waypointIndex++;
13        }
14    }
15    else
16    {
17        Destroy(gameObject);
18    }
19 }
20
```

↔ Vector2.MoveTowards

Vector2.MoveTowards(현재위치, 목표위치, 최대이동거리)

매 프레임 목표 지점을 향해 일정 거리만큼 이동합니다. 도착하면 다음 Waypoint로 전환합니다.

⚙️ ScriptableObject 생성

- 1 Waves and Paths 폴더에서
우클릭 → Create →
Scriptable Objects → WaveConf_SO
- 2 “Wave 0” 이름으로 생성
- 3 Inspector에서 설정:
 - Path Prefab: Path 0
 - Move Speed: 5

🔗 Enemy에 연결

- 1 Enemy 0 Prefab에 Pathfinder 스크립트 추가
- 2 Pathfinder의 **Wave Config**에 “Wave 0”
에셋을 드래그
- 3 실행하면 적이 Waypoint를 따라 이동!

💡 여러 Path

Path 0, Path 1, Path 2 ...를 만들고
각각 다른 Wave에 연결하면
다양한 이동 패턴을 만들 수 있습니다.

WaveConfigSO 확장 – 적 프리팹 리스트

</> WaveConfigSO에 적 프리팹 추가

```
1 public class WaveConfigSO : ScriptableObject
2 {
3     [SerializeField] List<GameObject> enemyPrefabs;
4     [SerializeField] Transform pathPrefab;
5     [SerializeField] float moveSpeed = 5f;
6
7     public int GetEnemyCount()
8     {
9         return enemyPrefabs.Count;
10    }
11
12    public GameObject GetEnemyPrefab(int index)
13    {
14        return enemyPrefabs[index];
15    }
16
17    // ... 기존 Getter 메서드들 ...
18 }
19
```

설명

WaveConfigSO 에셋의 Inspector에서 생성할 적 프리팹들을 리스트로 등록합니다. 리스트에 같은 프리팹을 여러 번 넣으면 그 수만큼 적이 생성됩니다.

EnemySpawner.cs – 적 생성기

</> EnemySpawner 기본 구조

```
1 public class EnemySpawner : MonoBehaviour
2 {
3     [SerializeField] WaveConfigSO currentWave;
4
5     void Start()
6     {
7         SpawnEnemies();
8     }
9
10    public WaveConfigSO GetCurrentWave()
11    {
12        // 는PathFinder 별도의 클currentWave 갖지 않고, 이를 불러서 쓰자
13        return currentWave;
14    }
15
16    void SpawnEnemies()
17    {
18        for (int i = 0; i < currentWave.GetEnemyCount(); i++)
19        {
20            Instantiate(currentWave.GetEnemyPrefab(i),
21                currentWave.GetStartingWaypoint().position,
22                Quaternion.identity,
23                transform);
24        }
25    }
26 }
27
```

주목: Instantiate와 Destroy

★ 주목 – Instantiate의 매개변수

```
1 Instantiate(prefab, position, rotation, parent);
```

```
2
```

prefab	복제할 원본 오브젝트 (Prefab)
--------	----------------------

position	생성될 월드 좌표 위치
----------	---------------------

rotation	회전값 (Quaternion.identity = 회전 없음)
----------	-----------------------------------

parent	부모 Transform (생략 가능). Hierarchy 정리에 유용
--------	--

★ 주목 – Destroy와 메모리 관리

- Destroy(gameObject) – 현재 오브젝트를 씬에서 제거
- Destroy(gameObject, 2f) – **2초 후에** 제거 (이펙트에 유용)
- Destroy하지 않으면 화면 밖 오브젝트가 계속 쌓여 **메모리 누수** 발생

주목: 스크립트 연결 누락 – 흔한 실수

⚠ Prefab에 스크립트를 연결했는지 확인!

Prefab에 스크립트를 **Add Component**하지 않으면, 코드를 아무리 잘 작성해도 동작하지 않습니다. 적이 보이지 않거나 움직이지 않을 때 가장 먼저 확인할 것:

- Enemy Prefab에 **PathFinder** 스크립트가 연결되어 있는가?
- Player Prefab에 **Player** 스크립트가 연결되어 있는가?
- EnemySpawner 오브젝트에 **EnemySpawner** 스크립트가 연결되어 있는가?

스크립트를 작성한 것과 Prefab에 연결하는 것은 **별개의 작업**입니다!

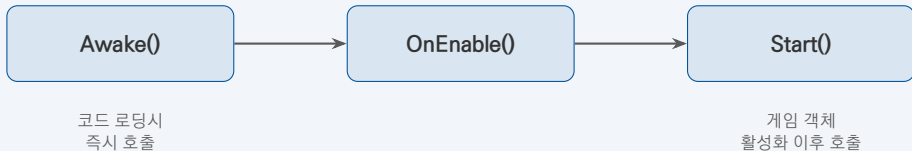
PathFinder.cs 변경 – EnemySpawner와 연동

</> Pathfinder 수정 버전

```
1 public class Pathfinder : MonoBehaviour
2 {
3     EnemySpawner enemySpawner;
4     WaveConfigSO waveConfig; // 여기서 Inspector 별도로 받지 못 하게 함
5     List<Transform> waypoints;
6     int waypointIndex = 0;
7
8     private void Awake()
9     {
10         enemySpawner = FindFirstObjectByType<EnemySpawner>();
11     }
12
13     void Start()
14     {
15         waveConfig = enemySpawner.GetCurrentWave();
16         waypoints = waveConfig.GetWaypoints();
17         transform.position = waypoints[waypointIndex].position;
18     }
19
20     void Update()
21     {
22         FollowPath();
23     }
24 }
25
```

Awake vs Start

🕒 Unity 생명주기: Awake와 Start의 차이



📌 왜 Awake에서 FindFirstObjectByType?

Awake()는 Start()보다 먼저 호출되므로, 다른 스크립트의 Start()에서 이 값을 안전하게 사용할 수 있습니다.
참조 설정은 Awake(), 초기화 로직은 Start()에!

Coroutine이란?

Coroutine 개념

- 시간이 걸리는 작업을 **중단**하고 다시 이어서 실행할 수 있게 해주는 함수
- 잠시 멈췄다가 다시 실행되는 함수
- 반환형: IEnumerator
- 호출: StartCoroutine()

기본 예제

```
1 IEnumerator WaitAndPrint()
2 {
3     Debug.Log("Start");
4     yield return
5         new WaitForSeconds(2f); // 초2 대기
6     Debug.Log("2 seconds passed");
7 }
8
9 // 호출
10 StartCoroutine(WaitAndPrint());
11
```

Sleep과의 차이

yield return은 **Coroutine만 멈추고** 게임의 다른 동작은 계속됩니다.
일반적인 Sleep/Thread.Sleep은 **모든 동작을 멈추므로** Unity에서 사용하면 안 됩니다.
→ **비동기(Asynchronous)** 동작 가능 (UI 반응이 멈추지 않음)

주목: Coroutine의 다양한 yield return

★ 주목 – yield return에 넘길 수 있는 것들

구문	동작
yield return null	다음 프레임 까지 대기
yield return new WaitForSeconds(n)	n초 동안 대기
yield return new WaitForEndOfFrame()	현재 프레임의 렌더링 완료 후 재개
yield return new WaitUntil(() => 조건)	조건이 true 가 될 때까지 대기
yield return StartCoroutine(다른것)	다른 Coroutine이 끝날 때까지 대기

★ 주목 – Coroutine 사용 시 주의사항

- Coroutine은 **MonoBehaviour**에서만 실행 가능 (ScriptableObject에서는 불가)
- GameObject가 **비활성화**되거나 **파괴**되면 Coroutine도 중단됨
- StopCoroutine() 또는 StopAllCoroutines()로 수동 중단 가능
- 무한 루프 Coroutine에는 반드시 yield return이 있어야 함 (없으면 Unity 멈춤!)

WaveConfigSO – 스폰 타이밍 추가

</> WaveConfigSO에 새로운 변수 추가

```
1 public class WaveConfigSO : ScriptableObject
2 {
3     [SerializeField] List<GameObject> enemyPrefabs;
4     [SerializeField] Transform pathPrefab;
5     [SerializeField] float moveSpeed = 5f;
6     [SerializeField] float timeBetweenEnemySpawns = 1f;
7     [SerializeField] float spawnTimeVariance = 0f;
8     [SerializeField] float minimumSpawnTime = 0.2f;
9
10    public float GetRandomSpawnTime()
11    {
12        float spawnTime = Random.Range(
13            timeBetweenEnemySpawns - spawnTimeVariance,
14            timeBetweenEnemySpawns + spawnTimeVariance);
15        return Mathf.Clamp(spawnTime, minimumSpawnTime, float.MaxValue);
16    }
17
18    // ... 기존 메서드들 ...
19 }
20
```

i 변수 설명

timeBetweenEnemySpawns: 적 생성 기본 간격 / spawnTimeVariance: 랜덤 변동폭 / minimumSpawnTime: 최소 생성 간격 (0 이하 방지)

EnemySpawner – Coroutine 적용

</> EnemySpawner를 Coroutine으로 수정

```
1 // using System.Collections 필요 (IEnumerator)
2 public class EnemySpawner : MonoBehaviour
3 {
4     [SerializeField] WaveConfigSO currentWave;
5
6     void Start() { StartCoroutine(SpawnEnemies()); }
7
8     IEnumerator SpawnEnemies()
9     {
10         for (int i = 0; i < currentWave.GetEnemyCount(); i++)
11         {
12             Instantiate(currentWave.GetEnemyPrefab(i),
13                 currentWave.GetStartingWaypoint().position,
14                 Quaternion.identity, transform);
15             yield return new WaitForSeconds(
16                 currentWave.GetRandomSpawnTime());
17         }
18     }
19 }
20
```



효과

적이 한꺼번에 생성되지 않고, 랜덤한 시간 간격을 두고 하나씩 등장합니다!

여러 Wave 사용하기

</> EnemySpawner – 다중 Wave 지원

```
1 public class EnemySpawner : MonoBehaviour
2 {
3     [SerializeField] List<WaveConfigSO> waveConfigs;
4     [SerializeField] float timeBetweenWaves = 0f;
5     WaveConfigSO currentWave;
6
7     void Start() { StartCoroutine(SpawnWaves()); }
8
9     public WaveConfigSO GetCurrentWave() { return currentWave; }
10
11     IEnumerator SpawnWaves()
12     {
13         foreach (WaveConfigSO wave in waveConfigs)
14         {
15             currentWave = wave;
16             for (int i = 0; i < currentWave.GetEnemyCount(); i++)
17             {
18                 Instantiate(currentWave.GetEnemyPrefab(i),
19                     currentWave.GetStartingWaypoint().position,
20                     Quaternion.identity, transform);
21                 yield return new WaitForSeconds(
22                     currentWave.GetRandomSpawnTime());
23             }
24             yield return new WaitForSeconds(timeBetweenWaves);
25         }
26     }
27 }
28
```

다중 Wave 설정

구조 정리

Waves and Paths 폴더에:

- Path 0, Path 1, Path 2 ... (경로 Prefab)
- Wave 0, Wave 1, Wave 2 ...
(WaveConfigSO 에셋)

각 Wave 에셋에서:

- 사용할 Path Prefab 지정
- 생성할 적 프리팹 리스트 설정
- 이동 속도, 스폰 간격 설정

EnemySpawner에 연결

EnemySpawner의 Inspector에서:

- **Wave Configs** 리스트에 Wave 0, Wave 1 등을 추가
- **Time Between Waves**: Wave 간 대기 시간

Coroutine의 특성 이해

SpawnWaves() 안에서 SpawnEnemies()를 별도 Coroutine으로 호출하면 어떻게 될까요?

→ Wave가 끝나기 전에 다음 Wave가 시작될 수 있습니다!

Wave 무한 반복 – do-while 루프

</> isLooping 플래그 추가 – SpawnWaves() 변경

```
1 // 애EnemySpawner [SerializeField] bool isLooping; 추가
2
3 IEnumerator SpawnWaves()
4 {
5     do
6     {
7         foreach (WaveConfigSO wave in waveConfigs)
8         {
9             currentWave = wave;
10            for (int i = 0; i < currentWave.GetEnemyCount(); i++)
11            {
12                Instantiate(currentWave.GetEnemyPrefab(i),
13                    currentWave.GetStartingWaypoint().position,
14                    Quaternion.identity, transform);
15                yield return new WaitForSeconds(
16                    currentWave.GetRandomSpawnTime());
17            }
18            yield return new WaitForSeconds(timeBetweenWaves);
19        }
20    }
21    while (isLooping);
22 }
23
```

무한 반복 설정

Inspector에서 설정

EnemySpawner의 **Is Looping** 체크박스를 활성화하면 모든 Wave가 끝난 후 처음부터 다시 반복합니다.

do-while vs while

do { ... } while(isLooping) - false여도 **최소 1번** 실행, true이면 **무한 반복**.
Coroutine의 yield return 덕분에 무한 루프여도 게임이 멈추지 않습니다!

이번 강의 정리

✓ 핵심 개념

- ❶ **New Input System**
Player Input 컴포넌트와 Input Actions로 입력 처리
- ❷ **Prefab**
게임 오브젝트의 재사용 가능한 템플릿
- ❸ **Viewport → World 좌표 변환**
화면 경계 내에 플레이어 가두기
- ❹ **Waypoint 시스템**
적의 이동 경로 정의

✓ 핵심 개념 (계속)

- ❺ **ScriptableObject**
데이터와 코드 분리, Wave 설정 관리
- ❻ **Coroutine**
시차를 두고 적 생성, 비동기 처리
- ❼ **Awake vs Start**
초기화 순서 제어
- ❽ **do-while 루프**
Wave 무한 반복

▶▶ 다음 강의 예고

전투 시스템 - 레이저 발사와 충돌 처리!

수고하셨습니다!

질문이 있으면 언제든지 물어보세요.